Code

```
#include <WiFi.h>
#include <WebServer.h>
#include <DHT.h>  // Include DHT sensor library

// ---------------- WiFi Credentials ----------------
const char* ssid     = "KALIDAS";
const char* password = "AvantikA!";

// ---------------- Login Credentials ----------------
const char* loginUser = "admin";
const char* loginPass = "1234";

// ---------------- Pin Config ----------------
#define SOIL_MOISTURE_PIN 34
#define PUMP_PIN 2
#define DHT_PIN 4       // GPIO pin for DHT11

// ---------------- DHT Sensor Setup ----------------
#define DHT_TYPE DHT11
DHT dht(DHT_PIN, DHT_TYPE);

bool isPumpOn = false;
bool isLoggedIn = false;
bool isCropSelected = false;
String selectedCrop = "None";

WebServer server(80);
```

```
// ---------------- Simulated Sensor Data (keeping for internal logic but not displaying) ----------
int simulatedPH = 6;  // placeholder
int simulatedN = 40;
int simulatedP = 20;
int simulatedK = 30;

// ---------------- Crop Thresholds ----------------
struct CropThreshold {
  int minMoisture, maxMoisture;
  float minPH, maxPH;
  int minN, minP, minK;
  int minTemp, maxTemp;  // Added temperature thresholds
  int minHumidity, maxHumidity;  // Added humidity thresholds
};

// Global threshold variables with temperature and humidity
CropThreshold wheat = {50, 70, 6.0, 7.5, 50, 30, 40, 15, 25, 40, 70};
CropThreshold rice  = {60, 80, 5.5, 6.5, 40, 20, 30, 20, 35, 70, 90};
CropThreshold tomato= {60, 80, 6.0, 6.8, 50, 25, 35, 18, 30, 50, 80};
CropThreshold potato= {65, 85, 5.0, 6.5, 60, 30, 40, 15, 25, 60, 85};
CropThreshold onion = {50, 70, 6.0, 7.0, 45, 20, 30, 10, 30, 50, 80};

// ---------------- Crop Information Database ----------------
struct CropInfo {
  String name;
  String description;
  String season;
  String growthPeriod;
  String waterRequirements;
```

```
  String fertilizers;
  String pesticides;
  String diseases;
  String harvesting;
  String tips;
};

CropInfo cropDatabase[] = {
  {
    "Wheat",
    "Wheat is a cereal grain grown for its seed, a staple food used to make flour, bread, pasta, pastry, etc.",
    "Rabi (Winter) Season - Oct to Mar",
    "110-130 days",
    "Moderate - 450-650 mm per cycle",
    "NPK (120:60:40 kg/ha), Zinc Sulphate (25 kg/ha), Urea (150 kg/ha)",
    "Neem-based pesticides, Chlorpyriphos for termites, Mancozeb for rust",
    "Rust, Smut, Karnal Bunt, Powdery Mildew",
    "When grains are hard and moisture content is 20-25%",
    "Rotate crops to prevent disease, Use certified seeds, Monitor for aphids"
  },
  {
    "Rice",
    "Rice is the seed of the grass species Oryza sativa (Asian rice) or Oryza glaberrima (African rice).",
    "Kharif (Monsoon) - Jun to Nov",
    "90-150 days depending on variety",
    "High - 1500-2000 mm per cycle",
    "NPK (100:50:50 kg/ha), Zinc Sulphate (25 kg/ha), Farmyard manure (10-12 tons/ha)",
```

"Carbofuran for stem borer, Monocrotophos for leaf folder, BPMC for brown plant hopper",

"Blast, Bacterial Blight, Sheath Blight, Tungro Virus",

"When 80-85% grains turn yellow and moisture is 20-25%",

"Maintain proper water level, Use resistant varieties, Practice integrated pest management"

},

{

"Tomato",

"Tomato is a flowering plant of the nightshade family, cultivated for its edible fruits.",

"Year-round with protection (main season: Oct-Feb)",

"70-90 days",

"Regular but moderate - 600-800 mm per cycle",

"NPK (150:80:80 kg/ha), Calcium Nitrate, Micronutrients (B, Zn, Fe)",

"Neem oil, Spinosad for caterpillars, Imidacloprid for whiteflies",

"Early Blight, Late Blight, Tomato Mosaic Virus, Blossom End Rot",

"When fruits are firm and fully colored, harvest every 3-4 days",

"Stake plants for better yield, Mulch to conserve moisture, Rotate with non-solanaceous crops"

},

{

"Potato",

"Potato is a root vegetable, a starchy tuber of the plant Solanum tuberosum.",

"Rabi (Oct-Jan) in plains, Year-round in hills",

"80-120 days",

"Moderate - 500-700 mm per cycle",

"NPK (150:100:100 kg/ha), Farmyard manure (20-25 tons/ha), Sulphur (20 kg/ha)",

"Chlorpyriphos for cutworms, Mancozeb for early blight, Metalaxyl for late blight",

"Late Blight, Early Blight, Black Scurf, Potato Virus Y",

"When leaves turn yellow and dry, 2-3 weeks after flowering",

```
    "Use disease-free seed potatoes, Hill soil around plants, Avoid waterlogging"
  },
  {
    "Onion",
    "Onion is a vegetable, the most widely cultivated species of the genus Allium.",
    "Rabi (Oct-Mar) for storage, Kharif (Jun-Oct) for fresh",
    "90-150 days",
    "Moderate - 350-550 mm per cycle",
    "NPK (120:60:60 kg/ha), Sulphur (45 kg/ha), Zinc Sulphate (25 kg/ha)",
    "Chlorpyriphos for thrips, Carbendazim for purple blotch, Neem cake for nematodes",
    "Purple Blotch, Stemphylium Blight, Basal Rot, Downy Mildew",
    "When tops fall over and neck tissues soften",
    "Plant in well-drained soil, Avoid excessive nitrogen, Cure properly before storage"
  }
};

// ---------------- Sensor Functions ----------------
int readSoilMoisture() {
  int rawValue = analogRead(SOIL_MOISTURE_PIN);
  int percentage = map(rawValue, 3500, 4095, 100, 0);
  return constrain(percentage, 0, 100);
}

float readTemperature() {
  return dht.readTemperature();  // Read temperature in Celsius
}

float readHumidity() {
  return dht.readHumidity();  // Read humidity percentage
```

```
}

// ---------------- Get Crop Thresholds ----------------
CropThreshold getCropThresholds(String cropName) {
  if (cropName == "Wheat") return wheat;
  else if (cropName == "Rice") return rice;
  else if (cropName == "Tomato") return tomato;
  else if (cropName == "Potato") return potato;
  else if (cropName == "Onion") return onion;
  // Return a default/safe threshold if not found
  return {40, 80, 6.0, 7.0, 30, 20, 20, 15, 30, 40, 80}; // Generic safe values
}

// ---------------- Soil and Environment Health Check ----------------
String checkSoilHealth(int soil, float ph, int N, int P, int K, float temp, float humidity) {
  CropThreshold th = getCropThresholds(selectedCrop);

  if (selectedCrop == "None") return "Please select a crop for tailored advice.";

  String msg = "";

  // Soil moisture check
  if (soil < th.minMoisture) msg += "Moisture too low<br>";
  else if (soil > th.maxMoisture) msg += "Moisture too high<br>";

  // pH check
  if (ph < th.minPH || ph > th.maxPH) msg += "pH not suitable<br>";

  // Nutrient checks
```

```
  if (N < th.minN) msg += "Low Nitrogen<br>";
  if (P < th.minP) msg += "Low Phosphorus<br>";
  if (K < th.minK) msg += "Low Potassium<br>";

  // Temperature check
  if (temp < th.minTemp) msg += "Temperature too low<br>";
  else if (temp > th.maxTemp) msg += "Temperature too high<br>";

  // Humidity check
  if (humidity < th.minHumidity) msg += "Humidity too low<br>";
  else if (humidity > th.maxHumidity) msg += "Humidity too high<br>";

  if (msg == "") msg = "Environment is suitable for " + selectedCrop;
  return msg;
}

// ---------- Get Crop Information ----------
CropInfo getCropInfo(String cropName) {
  for (int i = 0; i < 5; i++) {
    if (cropDatabase[i].name == cropName) {
      return cropDatabase[i];
    }
  }
  // Return a default crop info if not found (Wheat is index 0)
  return cropDatabase[0];
}

// ---------- HTML Pages with DHT11 Data ----------
String loginPage() {
```

```
return R"rawliteral(
<!DOCTYPE html>
<html>
<head>
 <title>Smart Irrigation - Login</title>
 <meta name="viewport" content="width=device-width, initial-scale=1.0">
 <style>
  * {
   margin: 0;
   padding: 0;
   box-sizing: border-box;
  }

  body {
   font-family: 'Segoe UI', Tahoma, Geneva, Verdana, sans-serif;
   background: linear-gradient(135deg, #667eea 0%, #764ba2 100%);
   min-height: 100vh;
   display: flex;
   justify-content: center;
   align-items: center;
   padding: 20px;
  }

  .login-container {
   background: rgba(255, 255, 255, 0.95);
   backdrop-filter: blur(10px);
   padding: 40px;
   border-radius: 20px;
   box-shadow: 0 20px 40px rgba(0, 0, 0, 0.1);
```

```
  width: 100%;
  max-width: 400px;
  text-align: center;
  border: 1px solid rgba(255, 255, 255, 0.2);
}

.logo {
  font-size: 2.5rem;
  margin-bottom: 10px;
  background: linear-gradient(135deg, #667eea, #764ba2);
  -webkit-background-clip: text;
  -webkit-text-fill-color: transparent;
}

h2 {
  color: #333;
  margin-bottom: 30px;
  font-weight: 600;
}

.input-group {
  margin-bottom: 20px;
  text-align: left;
}

label {
  display: block;
  margin-bottom: 8px;
  color: #555;
```

```
  font-weight: 500;
}

input {
  width: 100%;
  padding: 15px;
  border: 2px solid #e1e5e9;
  border-radius: 10px;
  font-size: 16px;
  transition: all 0.3s ease;
  background: #f8f9fa;
}

input:focus {
  outline: none;
  border-color: #667eea;
  background: white;
  box-shadow: 0 0 0 3px rgba(102, 126, 234, 0.1);
}

.login-btn {
  width: 100%;
  padding: 15px;
  background: linear-gradient(135deg, #667eea, #764ba2);
  color: white;
  border: none;
  border-radius: 10px;
  font-size: 16px;
  font-weight: 600;
```

```
      cursor: pointer;
      transition: transform 0.2s ease, box-shadow 0.2s ease;
      margin-top: 10px;
    }

    .login-btn:hover {
      transform: translateY(-2px);
      box-shadow: 0 10px 20px rgba(102, 126, 234, 0.3);
    }

    .footer {
      margin-top: 20px;
      color: #666;
      font-size: 14px;
    }
  </style>
 </head>
 <body>
  <div class="login-container">
   <div class="logo"> PLANT</div>
   <h2>Smart Irrigation System</h2>
   <form action="/login" method="POST">
    <div class="input-group">
     <label for="username">Username</label>
     <input type="text" id="username" name="username" placeholder="Enter your
username" required>
    </div>
    <div class="input-group">
     <label for="password">Password</label>
```

```
      <input type="password" id="password" name="password" placeholder="Enter your password" required>
      </div>
      <button type="submit" class="login-btn">Login to Dashboard</button>
    </form>
    <div class="footer">Secure Agricultural Monitoring</div>
  </div>
 </body>
 </html>
 )rawliteral";
}

String cropSelectionPage() {
 return R"rawliteral(
 <!DOCTYPE html>
 <html>
 <head>
  <title>Select Crop - Smart Irrigation</title>
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <style>
   * {
    margin: 0;
    padding: 0;
    box-sizing: border-box;
   }

   body {
    font-family: 'Segoe UI', Tahoma, Geneva, Verdana, sans-serif;
    background: linear-gradient(135deg, #f093fb 0%, #f5576c 100%);
    min-height: 100vh;
```

```
  display: flex;
  justify-content: center;
  align-items: center;
  padding: 20px;
}

.crop-container {
  background: rgba(255, 255, 255, 0.95);
  backdrop-filter: blur(10px);
  padding: 40px;
  border-radius: 20px;
  box-shadow: 0 20px 40px rgba(0, 0, 0, 0.1);
  width: 100%;
  max-width: 500px;
  text-align: center;
}

.logo {
  font-size: 2.5rem;
  margin-bottom: 10px;
}

h2 {
  color: #333;
  margin-bottom: 10px;
  font-weight: 600;
}

.subtitle {
```

```
    color: #666;
    margin-bottom: 30px;
  }

  .crop-select {
    width: 100%;
    padding: 15px;
    border: 2px solid #e1e5e9;
    border-radius: 10px;
    font-size: 16px;
    margin-bottom: 25px;
    background: #f8f9fa;
    appearance: none;
    background-image: url("data:image/svg+xml;charset=US-ASCII,<svg
xmlns='http://www.w3.org/2000/svg' viewBox='0 0 4 5'><path fill='%23666' d='M2 0L0
2h4zm0 5L0 3h4z'/></svg>");
    background-repeat: no-repeat;
    background-position: right 15px center;
    background-size: 12px;
  }

  .crop-select:focus {
    outline: none;
    border-color: #f5576c;
    box-shadow: 0 0 0 3px rgba(245, 87, 108, 0.1);
  }

  .select-btn {
    width: 100%;
    padding: 15px;
```

```
    background: linear-gradient(135deg, #f093fb, #f5576c);
    color: white;
    border: none;
    border-radius: 10px;
    font-size: 16px;
    font-weight: 600;
    cursor: pointer;
    transition: transform 0.2s ease, box-shadow 0.2s ease;
   }

   .select-btn:hover {
    transform: translateY(-2px);
    box-shadow: 0 10px 20px rgba(245, 87, 108, 0.3);
   }
  </style>
</head>
<body>
  <div class="crop-container">
   <div class="logo"> CROP</div>
   <h2>Select Your Crop</h2>
   <p class="subtitle">Choose the crop you're growing for customized monitoring</p>
   <form action="/selectCrop" method="POST">
    <select name="crop" class="crop-select" required>
     <option value="">Select a crop...</option>
     <option value="Wheat">Wheat</option>
     <option value="Rice">Rice</option>
     <option value="Tomato">Tomato</option>
     <option value="Potato">Potato</option>
     <option value="Onion">Onion</option>
```

```
      </select>
      <button type="submit" class="select-btn">Confirm Selection</button>
    </form>
  </div>
 </body>
 </html>
 )rawliteral";
}

String dashboardPage() {
 CropInfo currentCrop = getCropInfo(selectedCrop);
 CropThreshold currentThresholds = getCropThresholds(selectedCrop);

 String page = R"rawliteral(
 <!DOCTYPE html>
 <html>
 <head>
  <title>Dashboard - Smart Irrigation</title>
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <style>
   * {
    margin: 0;
    padding: 0;
    box-sizing: border-box;
   }

   body {
    font-family: 'Segoe UI', Tahoma, Geneva, Verdana, sans-serif;
    background: linear-gradient(135deg, #4facfe 0%, #00f2fe 100%);
```

```
  min-height: 100vh;
  padding: 20px;
}

.header {
  text-align: center;
  margin-bottom: 20px;
  color: white;
}

.logo {
  font-size: 2.5rem;
  margin-bottom: 10px;
}

h1 {
  font-size: 2rem;
  margin-bottom: 5px;
  text-shadow: 0 2px 4px rgba(0,0,0,0.1);
}

/* Desktop Navigation */
.nav-menu {
  display: flex;
  justify-content: center;
  gap: 8px;
  margin: 15px 0;
  flex-wrap: wrap;
}
```

```
.nav-btn {
  padding: 10px 15px;
  background: rgba(255, 255, 255, 0.2);
  color: white;
  border: none;
  border-radius: 20px;
  font-size: 13px;
  font-weight: 600;
  cursor: pointer;
  transition: all 0.3s ease;
  text-decoration: none;
}

.nav-btn:hover, .nav-btn.active {
  background: white;
  color: #4facfe;
  transform: translateY(-2px);
}

/* Mobile Navigation */
.mobile-nav-container {
  display: none;
  position: relative;
  margin: 15px 0;
}

.mobile-menu-btn {
  display: none;
```

```
  width: 100%;
  padding: 12px;
  background: rgba(255, 255, 255, 0.2);
  color: white;
  border: none;
  border-radius: 10px;
  font-size: 14px;
  font-weight: 600;
  cursor: pointer;
  transition: all 0.3s ease;
}

.mobile-nav-menu {
  display: none;
  flex-direction: column;
  background: rgba(255, 255, 255, 0.95);
  border-radius: 10px;
  margin-top: 5px;
  overflow: hidden;
  box-shadow: 0 5px 15px rgba(0,0,0,0.2);
}

.mobile-nav-btn {
  padding: 12px 15px;
  background: transparent;
  color: #333;
  border: none;
  border-bottom: 1px solid #eee;
  font-size: 13px;
```

```
  font-weight: 600;
  cursor: pointer;
  text-align: left;
  transition: all 0.3s ease;
  text-decoration: none;
}

.mobile-nav-btn:last-child {
  border-bottom: none;
}

.mobile-nav-btn:hover, .mobile-nav-btn.active {
  background: #4facfe;
  color: white;
}

.crop-info {
  background: rgba(255, 255, 255, 0.9);
  padding: 12px;
  border-radius: 12px;
  margin: 0 auto 15px;
  max-width: 400px;
  font-size: 1rem;
  font-weight: 600;
  color: #333;
  text-align: center;
}

.dashboard {
```

```
  display: grid;
  grid-template-columns: repeat(auto-fit, minmax(280px, 1fr));
  gap: 15px;
  margin-bottom: 20px;
}

.card {
  background: rgba(255, 255, 255, 0.95);
  backdrop-filter: blur(10px);
  padding: 20px;
  border-radius: 15px;
  box-shadow: 0 8px 25px rgba(0, 0, 0, 0.1);
  text-align: center;
  transition: transform 0.3s ease;
  border: 1px solid rgba(255, 255, 255, 0.2);
}

.card-value {
  font-size: 2rem;
  font-weight: 700;
  margin-bottom: 8px;
}

.card-desc {
  font-size: 0.9rem;
  color: #1C1C1E;
}

.moisture-value { color: #4facfe; }
```

```
.temp-value { color: #ff9f43; }
.humidity-value { color: #54a0ff; }

.pump-card {
  grid-column: span 2;
  background: linear-gradient(135deg, #667eea, #764ba2);
  color: white;
}

.pump-status {
  font-size: 1.3rem;
  font-weight: 600;
  margin: 10px 0;
}

.status-on { color: #00ff88; }
.status-off { color: #ff4757; }

.toggle-btn {
  padding: 10px 20px;
  background: rgba(255, 255, 255, 0.2);
  color: white;
  border: 2px solid white;
  border-radius: 20px;
  font-size: 0.9rem;
  font-weight: 600;
  cursor: pointer;
  transition: all 0.3s ease;
  text-decoration: none;
```

```
  display: inline-block;
}

.advisory-section {
  display: none;
  background: rgba(255, 255, 255, 0.95);
  padding: 20px;
  border-radius: 15px;
  margin-bottom: 20px;
  box-shadow: 0 8px 25px rgba(0, 0, 0, 0.1);
}

.advisory-section.active {
  display: block;
}

.advisory-title {
  font-size: 1.3rem;
  color: #333;
  margin-bottom: 15px;
}

.advisory-content {
  font-size: 1rem;
  line-height: 1.5;
  color: #555;
}

.info-grid {
```

```
  display: grid;
  grid-template-columns: repeat(auto-fit, minmax(200px, 1fr));
  gap: 15px;
  margin-top: 15px;
}

.info-item {
  background: #f8f9fa;
  padding: 15px;
  border-radius: 10px;
  border-left: 4px solid #4facfe;
}

.info-item h3 {
  color: #333;
  margin-bottom: 8px;
  font-size: 1.1rem;
}

.action-btn {
  padding: 10px 20px;
  margin: 0 8px;
  border: none;
  border-radius: 20px;
  font-size: 0.9rem;
  font-weight: 600;
  cursor: pointer;
  transition: all 0.3s ease;
  text-decoration: none;
```

```
  display: inline-block;
}

/* Mobile Responsive Styles */
@media (max-width: 768px) {
  body {
    padding: 10px;
  }

  .header {
    margin-bottom: 10px;
  }

  .logo {
     font-size: 2rem;
  }

  h1 {
    font-size: 1.5rem;
  }

  /* Hide desktop nav, show mobile nav */
  .nav-menu {
    display: none;
  }

  .mobile-nav-container {
    display: block;
  }
```

```
.mobile-menu-btn {
  display: block;
}

.dashboard {
  grid-template-columns: 1fr;
  gap: 10px;
}

.pump-card {
  grid-column: span 1;
}

.info-grid {
    grid-template-columns: 1fr;
}

.action-btn {
  display: block;
  width: 100%;
  margin: 5px 0;
  text-align: center;
}

.nav-buttons {
  display: flex;
  flex-direction: column;
  gap: 10px;
```

```
    }
   }

   @media (max-width: 480px) {
    .card {
     padding: 15px;
    }

    .card-value {
     font-size: 1.5rem;
    }

    .advisory-section {
     padding: 15px;
    }

    .advisory-title {
     font-size: 1.1rem;
    }
   }
  </style>
</head>
<body>
  <div class="header">
   <div class="logo"> DASHBOARD</div>
   <h1>Smart Irrigation Dashboard</h1>
   <div class="crop-info">Current Crop: )rawliteral" + selectedCrop + R"rawliteral(</div>

   <!-- Desktop Navigation -->
```

```
    <div class="nav-menu">
      <button class="nav-btn active" onclick="showSection('dashboard', this)"> Live Data</button>
      <button class="nav-btn" onclick="showSection('cropInfo', this)"> Crop Info</button>
      <button class="nav-btn" onclick="showSection('fertilizers', this)"> Fertilizers</button>
      <button class="nav-btn" onclick="showSection('pesticides', this)"> Pesticides</button>
      <button class="nav-btn" onclick="showSection('diseases', this)"> Diseases</button>
      <button class="nav-btn" onclick="showSection('harvesting', this)"> Harvesting</button>
    </div>

    <!-- Mobile Navigation -->
    <div class="mobile-nav-container">
      <button class="mobile-menu-btn" onclick="toggleMobileMenu()">
        Menu
      </button>
      <div class="mobile-nav-menu" id="mobileNavMenu">
        <button class="mobile-nav-btn active" onclick="showSection('dashboard', this); hideMobileMenu();"> Live Data</button>
        <button class="mobile-nav-btn" onclick="showSection('cropInfo', this); hideMobileMenu();"> Crop Info</button>
        <button class="mobile-nav-btn" onclick="showSection('fertilizers', this); hideMobileMenu();"> Fertilizers</button>
        <button class="mobile-nav-btn" onclick="showSection('pesticides', this); hideMobileMenu();"> Pesticides</button>
        <button class="mobile-nav-btn" onclick="showSection('diseases', this); hideMobileMenu();"> Diseases</button>
        <button class="mobile-nav-btn" onclick="showSection('harvesting', this); hideMobileMenu();"> Harvesting</button>
      </div>
    </div>
```

```
  </div>


  <div id="dashboard" class="advisory-section active">
   <div class="dashboard">
    <div class="card">
     <div class="card-icon"></div>
     <div class="card-title">Soil Moisture</div>
     <div class="card-value moisture-value" id="moisture">--%</div>
     <div class="card-desc">Optimal: )rawliteral" +
String(currentThresholds.minMoisture) + "-" + String(currentThresholds.maxMoisture) +
R"rawliteral(%</div>
    </div>

    <div class="card">
     <div class="card-icon"></div>
     <div class="card-title">Temperature</div>
     <div class="card-value temp-value" id="temperature">--°C</div>
     <div class="card-desc">Optimal: )rawliteral" + String(currentThresholds.minTemp) +
"-" + String(currentThresholds.maxTemp) + R"rawliteral(°C</div>
    </div>

    <div class="card">
     <div class="card-icon"></div>
     <div class="card-title">Humidity</div>
     <div class="card-value humidity-value" id="humidity">--%</div>
     <div class="card-desc">Optimal: )rawliteral" +
String(currentThresholds.minHumidity) + "-" + String(currentThresholds.maxHumidity) +
R"rawliteral(%</div>
    </div>

    <div class="card pump-card">
```

```
        <div class="card-icon"></div>
        <div class="card-title">Water Pump</div>
        <div class="pump-status" id="pumpStatus">Status: <span id="pumpText">--
</span></div>
        <a href='/toggle' class="toggle-btn" id="pumpBtn">Toggle Pump</a>
        <div class="card-desc">Auto-mode: Moisture < 40%</div>
       </div>
     </div>

     <div class="advisory-section active">
       <div class="advisory-title">Environment Health Advisory</div>
       <div class="advisory-content" id="advisory">Loading recommendations...</div>
     </div>
   </div>

   <div id="cropInfo" class="advisory-section">
     <div class="advisory-title"> About )rawliteral" + selectedCrop + R"rawliteral(</div>
     <div class="advisory-content">
       <p><strong>Description:</strong> )rawliteral" + currentCrop.description +
R"rawliteral(</p>
       <div class="info-grid">
         <div class="info-item">
           <h3> Growing Season</h3>
           <p>)rawliteral" + currentCrop.season + R"rawliteral(</p>
         </div>
         <div class="info-item">
           <h3> Growth Period</h3>
           <p>)rawliteral" + currentCrop.growthPeriod + R"rawliteral(</p>
         </div>
         <div class="info-item">
```

```
          <h3> Water Requirements</h3>
          <p>)rawliteral" + currentCrop.waterRequirements + R"rawliteral(</p>
        </div>
        <div class="info-item">
          <h3> Pro Tips</h3>
          <p>)rawliteral" + currentCrop.tips + R"rawliteral(</p>
        </div>
      </div>
    </div>
  </div>

  <div id="fertilizers" class="advisory-section">
    <div class="advisory-title"> Fertilizer Recommendations for )rawliteral" + selectedCrop
+ R"rawliteral(</div>
    <div class="advisory-content">
      <div class="info-item">
        <h3>Recommended Fertilizers</h3>
        <p>)rawliteral" + currentCrop.fertilizers + R"rawliteral(</p>
      </div>
      <div class="info-grid">
        <div class="info-item">
          <h3> Application Timing</h3>
          <p>Apply fertilizers in split doses: basal dose at planting, then top dressing during
growth stages</p>
        </div>
        <div class="info-item">
          <h3> Precautions</h3>
          <p>Test soil before application, avoid over-fertilization, maintain proper moisture
after application</p>
        </div>
```

```
      </div>
    </div>
  </div>

  <div id="pesticides" class="advisory-section">
    <div class="advisory-title"> Pest Management for )rawliteral" + selectedCrop +
R"rawliteral(</div>
    <div class="advisory-content">
      <div class="info-item">
        <h3>Recommended Pesticides</h3>
        <p>)rawliteral" + currentCrop.pesticides + R"rawliteral(</p>
      </div>
      <div class="info-grid">
        <div class="info-item">
          <h3> Preventive Measures</h3>
          <p>Use resistant varieties, practice crop rotation, maintain field sanitation</p>
        </div>
        <div class="info-item">
          <h3> Monitoring</h3>
          <p>Regular field inspection, use pheromone traps, monitor weather conditions</p>
        </div>
      </div>
    </div>
  </div>

  <div id="diseases" class="advisory-section">
    <div class="advisory-title"> Common Diseases in )rawliteral" + selectedCrop +
R"rawliteral(</div>
    <div class="advisory-content">
      <div class="info-item">
```

```
        <h3>Major Diseases</h3>
        <p>)rawliteral" + currentCrop.diseases + R"rawliteral(</p>
       </div>
       <div class="info-grid">
        <div class="info-item">
         <h3> Prevention</h3>
         <p>Use disease-free seeds, practice proper spacing, avoid waterlogging</p>
        </div>
        <div class="info-item">
         <h3> Treatment</h3>
         <p>Apply fungicides at first symptoms, remove infected plants, improve air
circulation</p>
        </div>
       </div>
     </div>
   </div>

   <div id="harvesting" class="advisory-section">
    <div class="advisory-title"> Harvesting Guide for )rawliteral" + selectedCrop +
R"rawliteral(</div>
    <div class="advisory-content">
     <div class="info-item">
      <h3>Harvesting Indicators</h3>
      <p>)rawliteral" + currentCrop.harvesting + R"rawliteral(</p>
     </div>
     <div class="info-grid">
      <div class="info-item">
       <h3> Best Time</h3>
       <p>Harvest in morning hours, avoid rainy days, use proper tools</p>
      </div>
```

```
        <div class="info-item">
          <h3> Post-Harvest</h3>
          <p>Proper cleaning, grading, storage in cool dry place, monitor for storage
pests</p>
        </div>
      </div>
    </div>
  </div>

  <div class="nav-buttons">
    <a href='/changeCrop' class="action-btn change-crop-btn"> Change Crop</a>
    <a href='/logout' class="action-btn logout-btn"> Logout</a>
  </div>

  <script>
    function showSection(sectionId, element) {
      document.querySelectorAll('.advisory-section').forEach(section => {
        section.classList.remove('active');
      });

      document.getElementById(sectionId).classList.add('active');

      document.querySelectorAll('.nav-btn').forEach(btn => {
        btn.classList.remove('active');
      });

      document.querySelectorAll('.mobile-nav-btn').forEach(btn => {
        btn.classList.remove('active');
      });
```

```
  if (element) {
    element.classList.add('active');
  }
 }

 function toggleMobileMenu() {
  const mobileMenu = document.getElementById('mobileNavMenu');
  mobileMenu.style.display = mobileMenu.style.display === 'flex' ? 'none' : 'flex';
 }

 function hideMobileMenu() {
  document.getElementById('mobileNavMenu').style.display = 'none';
 }

 // Close mobile menu when clicking outside
 document.addEventListener('click', function(event) {
  const mobileNavContainer = document.querySelector('.mobile-nav-container');
  const mobileMenuBtn = document.querySelector('.mobile-menu-btn');
  const mobileNavMenu = document.getElementById('mobileNavMenu');

  if (!mobileNavContainer.contains(event.target) && mobileNavMenu.style.display === 'flex') {
    hideMobileMenu();
  }
 });

 function updateDashboard(data) {
  // Update sensor values
  document.getElementById('moisture').textContent = data.soil + '%';
  document.getElementById('temperature').textContent = data.temperature + '°C';
```

```
  document.getElementById('humidity').textContent = data.humidity + '%';

  // Update pump status
  const pumpText = document.getElementById('pumpText');
  const pumpBtn = document.getElementById('pumpBtn');

  if (data.pump === 'true') {
   pumpText.textContent = 'RUNNING';
   pumpText.className = 'status-on';
   pumpBtn.textContent = 'Turn Off Pump';
  } else {
   pumpText.textContent = 'STOPPED';
   pumpText.className = 'status-off';
   pumpBtn.textContent = 'Turn On Pump';
  }

  // Update advisory
  document.getElementById('advisory').innerHTML = data.advisory;

  // Update background based on moisture level
  updateBackground(data.soil);
}

function updateBackground(moisture) {
  const body = document.body;
  if (moisture < 30) {
   body.style.background = 'linear-gradient(135deg, #ff6b6b 0%, #ff8e8e 100%)';
  } else if (moisture < 60) {
   body.style.background = 'linear-gradient(135deg, #4facfe 0%, #00f2fe 100%)';
```

```
      } else {
        body.style.background = 'linear-gradient(135deg, #1dd1a1 0%, #00d2d3 100%)';
      }
    }

    function fetchData() {
      fetch('/data')
        .then(response => response.json())
        .then(data => updateDashboard(data))
        .catch(error => console.error('Error fetching data:', error));
    }

    // Fetch data immediately and then every 3 seconds
    fetchData();
    setInterval(fetchData, 3000);
  </script>
 </body>
 </html>
 )rawliteral";
 return page;
}

// ---------------- Handlers ----------------
void handleRoot() {
 if (!isLoggedIn) server.send(200, "text/html", loginPage());
 else if (!isCropSelected) server.send(200, "text/html", cropSelectionPage());
 else server.send(200, "text/html", dashboardPage());
}
```

```
void handleLogin() {
  if (server.method() == HTTP_POST) {
    String user = server.arg("username");
    String pass = server.arg("password");
    if (user == loginUser && pass == loginPass) {
      isLoggedIn = true;
      server.sendHeader("Location", "/");
      server.send(303);
      return;
    }
  }
  server.send(200, "text/html", "<div style='text-align:center; padding:50px; font-family: Arial;'><h2 style='color:red;'>Login Failed!</h2><p>Invalid credentials</p><a href='/' style='padding:10px 20px; background:#667eea; color:white; text-decoration:none; border-radius:5px;'>Try Again</a></div>");
}

void handleCropSelection() {
  if (server.method() == HTTP_POST) {
    selectedCrop = server.arg("crop");
    isCropSelected = true;
    server.sendHeader("Location", "/");
    server.send(303);
  } else
  {
    server.send(200, "text/html", cropSelectionPage());
  }
}

void handleTogglePump() {
```

```
  isPumpOn = !isPumpOn;
  digitalWrite(PUMP_PIN, isPumpOn ? HIGH : LOW);
  server.sendHeader("Location", "/");
  server.send(303);
}

void handleChangeCrop() {
  isCropSelected = false;
  selectedCrop = "None";
  server.sendHeader("Location", "/");
  server.send(303);
}

void handleLogout() {
  isLoggedIn = false;
  isCropSelected = false;
  selectedCrop = "None";
  isPumpOn = false;
  digitalWrite(PUMP_PIN, LOW);
  server.sendHeader("Location", "/");
  server.send(303);
}

void handleData() {
  int soil = readSoilMoisture();
  float temperature = readTemperature();
  float humidity = readHumidity();

  String advisory = checkSoilHealth(soil, simulatedPH, simulatedN, simulatedP, simulatedK,
temperature, humidity);
```

```
  // Replace <br> with \n for JSON
  advisory.replace("<br>", "\\n");

  String json = "{";
  json += "\"soil\":" + String(soil) + ",";
  json += "\"temperature\":" + String(temperature, 1) + ",";  // 1 decimal place
  json += "\"humidity\":" + String(humidity, 1) + ",";       // 1 decimal place
  json += "\"pump\":\"" + String(isPumpOn ? "true" : "false") + "\",";
  json += "\"advisory\":\"" + advisory + "\"";
  json += "}";

  server.send(200, "application/json", json);
}

void handleNotFound() {
  server.send(404, "text/html", "<div style='text-align:center; padding:50px; font-family: Arial;'><h2 style='color:red;'>404 - Page Not Found</h2><a href='/' style='padding:10px 20px; background:#667eea; color:white; text-decoration:none; border-radius:5px;'>Go Home</a></div>");
}

// ---------------- Setup ----------------
void setup() {
  Serial.begin(115200);

  // Initialize pins
  pinMode(SOIL_MOISTURE_PIN, INPUT);
  pinMode(PUMP_PIN, OUTPUT);
  digitalWrite(PUMP_PIN, LOW);
```

```
// Initialize DHT sensor
dht.begin();
Serial.println("DHT11 sensor initialized");

// Connect to WiFi
Serial.print("Connecting to ");
Serial.println(ssid);
WiFi.begin(ssid, password);

int attempts = 0;
while (WiFi.status() != WL_CONNECTED && attempts < 20) {
  delay(1000);
  Serial.print(".");
  attempts++;
}

if (WiFi.status() == WL_CONNECTED) {
  Serial.println("\nWiFi connected!");
  Serial.print("IP address: ");
  Serial.println(WiFi.localIP());
} else {
  Serial.println("\nFailed to connect to WiFi");
}

// Set up server routes
server.on("/", handleRoot);
server.on("/login", handleLogin);
server.on("/selectCrop", handleCropSelection);
```

```
  server.on("/toggle", handleTogglePump);
  server.on("/changeCrop", handleChangeCrop);
  server.on("/logout", handleLogout);
  server.on("/data", handleData);
  server.onNotFound(handleNotFound);

  server.begin();
  Serial.println("HTTP server started");
}

// ---------------- Main Loop ----------------
void loop() {
  server.handleClient();

  // Auto pump control based on soil moisture
  CropThreshold currentThresholds = getCropThresholds(selectedCrop);

  int soil = readSoilMoisture();

  int autoMinMoisture = isCropSelected ? currentThresholds.minMoisture : 40;
  int autoMaxMoisture = isCropSelected ? currentThresholds.maxMoisture : 80;

  if (soil < autoMinMoisture && !isPumpOn) {
    isPumpOn = true;
    digitalWrite(PUMP_PIN, HIGH);
    Serial.println("Auto: Pump turned ON - Low moisture");
  } else if (soil > autoMaxMoisture && isPumpOn) {
    isPumpOn = false;
    digitalWrite(PUMP_PIN, LOW);
```

```
  Serial.println("Auto: Pump turned OFF - High moisture");
 }

 // Print sensor readings to serial for debugging
 static unsigned long lastSensorRead = 0;
 if (millis() - lastSensorRead > 10000) {  // Print every 10 seconds
  lastSensorRead = millis();
  Serial.print("Temperature: ");
  Serial.print(readTemperature());
  Serial.print("°C, Humidity: ");
  Serial.print(readHumidity());
  Serial.print("%, Soil Moisture: ");
  Serial.print(soil);
  Serial.println("%");
 }

 delay(1000);
}
```